

ПМ3 Разработка модулей ПО.

РО 3.1 Понимать и применять принципы объектно-ориентированного и асинхронного программирования.

Тема 2. Асинхронно программирование.

Лекция 13. Цепочки промисов и Promise.all.

Цель занятия:

Понять, как использовать цепочки промисов для последовательной обработки асинхронных операций и как обрабатывать несколько параллельных операций с помощью Promise.all.

Учебные вопросы:

- 1. Что такое цепочка промисов и как она работает?**
- 2. Promise.all. Обработка нескольких параллельных промисов.**
- 3. Обработка ошибок в цепочках промисов.**
- 4. Promise.race, Promise.allSettled, Promise.any.**

1. Что такое цепочка промисов и как она работает?

Promise — это объект в JavaScript, который представляет результат асинхронной операции:

- ожидание (pending),
- успех (fulfilled),
- ошибка (rejected).

Чтобы обработать результат, используется метод `.then()` (для успешного результата) и `.catch()` (для ошибки).

Цепочка промисов — это последовательность вызовов `.then()`, где каждый следующий `.then()` получает результат из предыдущего.

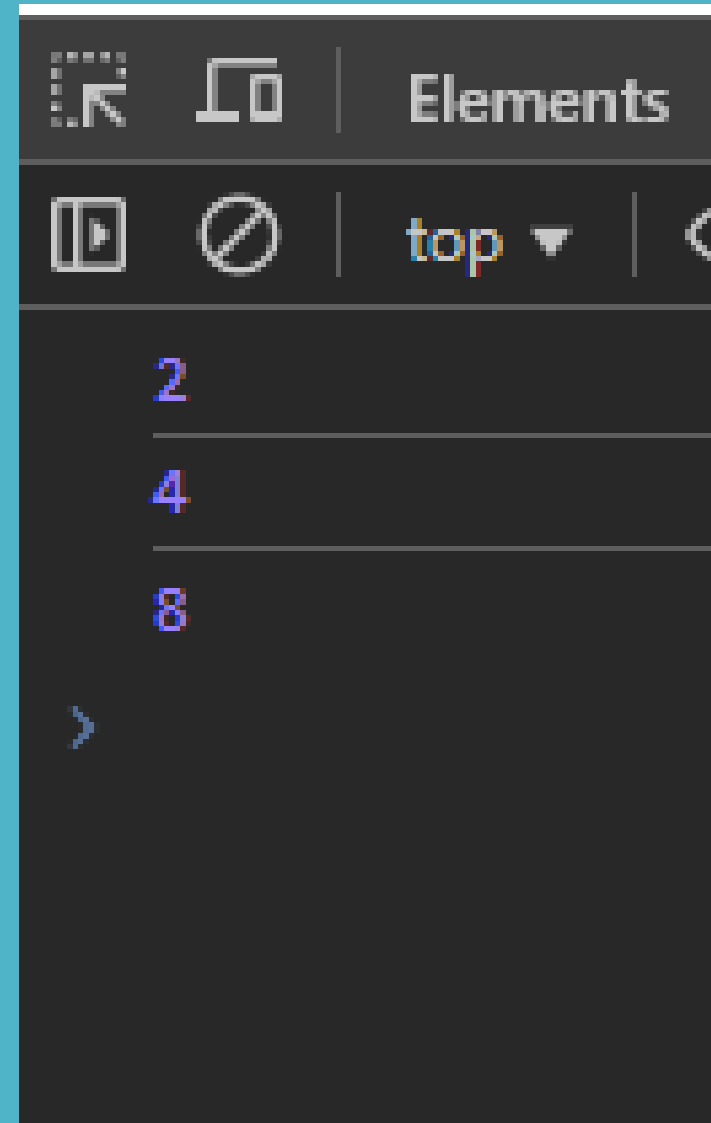
Таким образом можно выполнять несколько асинхронных операций по порядку, не погружаясь в «ад колбэков».

Принцип работы:

- Каждый вызов `.then()` возвращает новый промис.
- Результат функции внутри `.then()` передаётся в следующий `.then()`.
- Если внутри `.then()` возвращается обычное значение → оно автоматически «оборачивается» в промис.
- Если возвращается другой промис → выполнение «ждёт» его завершения.

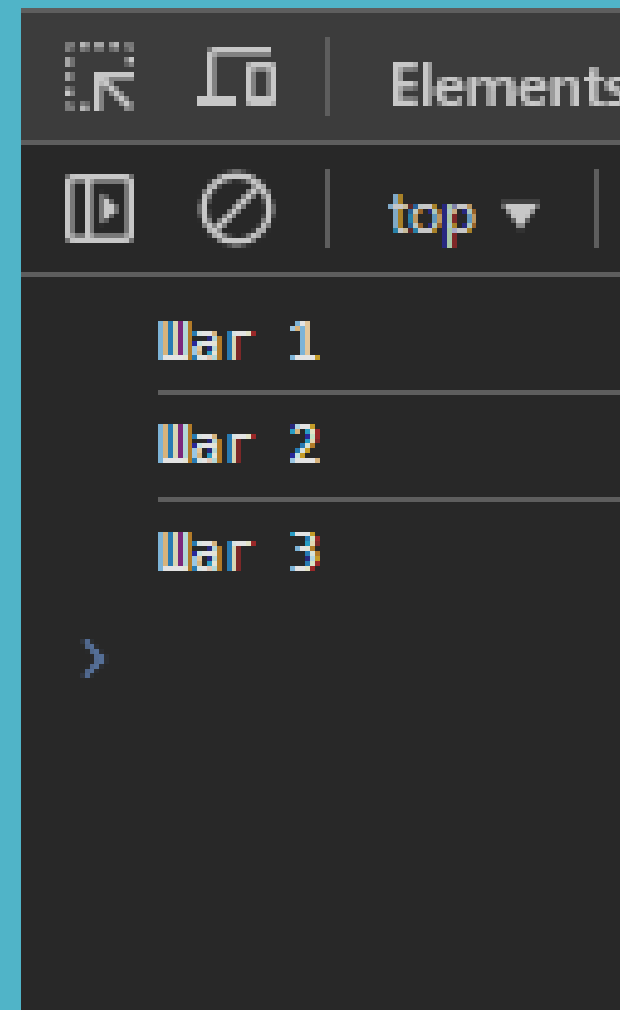
Пример цепочки. Здесь каждый `.then()` получает результат из предыдущего и передаёт его дальше.

```
new Promise((resolve) => {
  resolve(2);
})
.then((num) => {
  console.log(num); // 2
  return num * 2;
})
.then((num) => {
  console.log(num); // 4
  return num * 2;
})
.then((num) => {
  console.log(num); // 8
});
```



Асинхронный пример. Здесь шаги выполняются последовательно, несмотря на асинхронные задержки.

```
function delay(ms) {  
  return new Promise((resolve) => setTimeout(resolve, ms));  
}  
  
delay(1000)  
  .then(() => {  
    console.log("War 1");  
    return delay(1000);  
  })  
  .then(() => {  
    console.log("War 2");  
    return delay(1000);  
  })  
  .then(() => {  
    console.log("War 3");  
  });
```



Зачем нужны цепочки:

- Для последовательного выполнения асинхронных задач.
- Чтобы упростить код и избежать вложенных колбэков («callback hell»).
- Для удобной передачи результатов от одной операции к следующей.

 Вывод:

Цепочка промисов — это механизм, позволяющий выполнять асинхронные операции шаг за шагом, передавая результат от одного `.then()` к другому.

Каждый `.then()` создаёт новый промис и может возвращать как значение, так и другой промис.

2. Promise.all. Обработка нескольких параллельных промисов.

Часто нужно запустить несколько асинхронных операций **параллельно** (например, загрузить данные с разных серверов или выполнить несколько задержек), а затем дождаться, когда все они завершатся, прежде чем продолжить.

Обычная цепочка промисов (.then) решает только последовательное выполнение. Для параллельных операций она неудобна.

Что такое Promise.all?

Promise.all — это статический метод класса Promise, который принимает массив (или другой итерируемый объект) промисов и возвращает новый промис.

Если все промисы успешно завершились, то возвращается массив их результатов в том же порядке, в котором они были переданы.

Если хотя бы один промис завершился с ошибкой, весь Promise.all завершается с ошибкой (reject).

Синтаксис:

```
Promise.all([promise1, promise2, promise3])  
  .then((results) => {  
    console.log(results); // массив результатов  
  })  
  .catch((error) => {  
    console.error(error); // ошибка, если хотя бы один промис отклонён  
  });
```

Пример работы. Три асинхронные операции выполняются параллельно. Результат вернётся только тогда, когда завершатся все три задержки.

```
function delay(ms, value) {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve(value), ms);  
  });  
}  
Promise.all([  
  delay(1000, "A"),  
  delay(2000, "B"),  
  delay(1500, "C")  
])  
  .then((results) => {  
    console.log(results); // ["A", "B", "C"]  
  });
```

Когда полезно использовать Promise.all:

- Загрузка нескольких независимых ресурсов (например, разные API-запросы).
- Одновременный запуск нескольких асинхронных вычислений.
- Ожидание группы промисов, когда нужно продолжить только после их общего завершения.

Особенности:

- Порядок результатов сохраняется (по порядку промисов в массиве, а не по времени их выполнения).
- Если один промис завершился с ошибкой → весь Promise.all возвращает эту ошибку.
- Если нужно дождаться выполнения всех промисов, даже с ошибками, используют Promise.allSettled.

 Вывод: Promise.all позволяет запускать несколько промисов параллельно.

3. Обработка ошибок в цепочках промисов.

Ошибки в Promise.all:

- Если хотя бы один промис завершится с ошибкой, весь Promise.all переходит в состояние rejected.
- Ошибка будет передана в .catch().
- Успешные результаты других промисов при этом игнорируются.

Пример:

```
function delay(ms, value, isFail = false) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      if (isFail){  
        reject(new Error("Error " + value))  
      } else {  
        resolve(value)  
      }  
    }, ms)  
  })  
}
```

В этом примере в `.catch()` попадёт ошибка из второго промиса, и весь `Promise.all` завершится с ней.

```
const arr = [  
  delay(1000, 111),  
  delay(3000, 222),  
  delay(2000, 333, true),  
  delay(4000, 444),  
  delay(1000, 555),  
];  
  
Promise.all(arr)  
  .then((res) => console.log(res))  
  .catch((e) => console.log(e.message))
```

```
● PS C:\Users\user\Documents\oop> node index.js  
● Error 333
```

Promise.allSettled — это «докладываю по каждому». Он дожждется завершения всех промисов (и успешных, и провальных) и вернет массив объектов, описывающих судьбу каждого из них.

```
Promise.allSettled(arr)
  .then((res) => console.log(res))
  .catch((e) => console.log(e.message))
```

Результат:

```
PS C:\Users\user\Documents\oop> node index.js
[
  { status: 'fulfilled', value: 111 },
  { status: 'fulfilled', value: 222 },
  {
    status: 'rejected',
    reason: Error: Error 333
      at Timeout._onTimeout (C:\Users\user\Documents\oop\index.js:5:20)
      at listOnTimeout (node:internal/timers:588:17)
      at process.processTimers (node:internal/timers:523:7)
  },
  { status: 'fulfilled', value: 444 },
  { status: 'fulfilled', value: 555 }
]
```

Пример с фильтрацией:

```
Promise.allSettled(arr).then((res) => {  
  const successful = res  
    .filter((res) => res.status === "fulfilled")  
    .map((res) => res.value);  
  console.log("Успешно загружено:", successful);  
  const errors = res  
    .filter((res) => res.status === "rejected")  
    .map((res) => res.reason.message)  
  console.log("Загружено с ошибками:", errors);  
});
```

Успешно загружено: [111, 222, 444, 555]

Загружено с ошибками: ['Error 333']

Вывод:

- При использовании `Promise.all` учитывайте, что падение одного промиса обрушит весь результат.
- Для «устойчивых» сценариев (например, загрузка сразу многих данных, где можно пропустить часть ошибок) применяйте `Promise.allSettled`.

4. Promise.race, Promise.allSettled, Promise.any.

Promise.race.

- Принимает массив (или другой итерируемый объект) промисов.
- Возвращает первый завершившийся промис — не важно, был он успешным или с ошибкой.

Пример:

```
const arr = [  
  delay(1000, 111),  
  delay(3000, 222),  
  delay(2000, 333, true),  
  delay(4000, 444),  
  delay(500, 555, true),  
];  
  
Promise.race(arr)  
  .then((res) => console.log("Результат:", res))  
  .catch((e) => console.log(e.message));
```

Используется, когда нужно получить **самый быстрый результат** (например, при выборе самого быстрого сервера).

Promise.any:

- Принимает массив промисов.
- Возвращает первый успешно завершившийся промис.
- Ошибки игнорируются, пока хотя бы один промис не выполнится успешно.
- Если все промисы завершились с ошибкой → возвращает ошибку типа `AggregateError`.

Используется, когда нужно дождаться **первого успешного результата**, а остальные можно игнорировать (например, когда делаем несколько запросов на разные источники данных и берём первый успешный).

Пример:

```
const arr = [  
  delay(1000, 111),  
  delay(3000, 222),  
  delay(2000, 333, true),  
  delay(4000, 444),  
  delay(500, 555, true),  
];  
  
Promise.any(arr)  
  .then((res) => console.log("Результат:", res))  
  .catch((e) => console.log(e.message));
```

Сравнение методов:

Метод	Успешное завершение	Ошибка
<code>Promise.all</code>	Все промисы успешны → массив результатов	Один промис упал → вся операция с ошибкой
<code>Promise.race</code>	Первый завершившийся (успех или ошибка)	Ошибка, если первым завершился с ошибкой
<code>Promise.allSettled</code>	Всегда успешно, массив статусов	Ошибки не прерывают выполнение
<code>Promise.any</code>	Первый успешный промис	Ошибка, если все упали (AggregateError)

✓ Вывод (итоговое сравнение):

- **Promise.all** — ждём, пока выполнятся **все промисы успешно**.
Если хотя бы один промис упадёт → весь результат будет ошибкой.
- **Promise.race** — ждём **первый завершившийся промис** (успех или ошибка).
- **Promise.allSettled** — ждём завершения **всех промисов**, получаем массив статусов (*успех/ошибка*), ошибки не прерывают выполнение.
- **Promise.any** — ждём **первый успешный результат**, ошибки игнорируем, пока не упадут все (тогда будет `AggregateError`).

Контрольные вопросы:

- Что произойдёт, если хотя бы один промис в `Promise.all` завершится с ошибкой?
- Чем отличается `Promise.race` от `Promise.all`?
- В чём особенность `Promise.allSettled`? Когда его стоит использовать?
- Чем отличается `Promise.any` от `Promise.race`?
- Что произойдёт, если все промисы в `Promise.any` завершатся с ошибкой?

Материалы лекций:

<https://github.com/ShViktor72/Education>

Обратная связь:

colledge20education23@gmail.com